



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

INGENIERÍA INFORMÁTICA

LENGUAJES Y COMPILADORES

"Arquitectura de Lenguajes y Compiladores: Paradigmas, Benchmarking Estructural y Diseño Automático de Sistemas Físico- Lógicos"

Profesor:

Félix Márquez

GRUPO NABLA:

Daniel Villalba Santamaría - V27.506.542

Luis Millán – V30040213

Isabella Rodríguez - V31.521.612

Andrés Gómez - V31.085.717

Ciudad Guayana, Junio del 2026

Resumen académico

El presente informe de investigación aborda la infraestructura lingüística y computacional de las herramientas de desarrollo modernas, evaluando el lenguaje como un producto de software minuciosamente diseñado bajo restricciones formales rigurosas. La metodología se estructuró mediante una cadena de dependencias técnicas distribuidas en cuatro roles interconectados. Inicialmente, el Arquitecto de Paradigmas estableció la fundamentación teórica de los modelos Imperativo, Orientado a Objetos, Funcional, Lógico y Concurrente/Actores, analizando la convergencia multiparadigma contemporánea. Esta base conceptual sirvió como justificación directa para que el Ingeniero de Benchmarking sometiera a pruebas empíricas de procesamiento intensivo a cuatro entornos tecnológicos diferenciados: Zig, Rust, Python y JavaScript, estructurando una matriz formal de tiempos de ejecución promedio y consumo de memoria pico. Posteriormente, el Analista Gramatical deconstruye dichos códigos funcionales para analizar microscópicamente su morfología léxica y graficar la jerarquía sintáctica de sus estructuras de control esenciales. Esta extracción de reglas proveyó el molde estructural indispensable para que el Diseñador del DSL ejecutará la ingeniería creativa del denominado Lenguaje L, orientado al control automatizado del sistema de micro redes eléctricas inteligentes ECO-GRID. La especificación formal del nuevo lenguaje abarca su alfabeto, tokens y una gramática sintáctica abstracta unificada, validando su expresividad y coherencia lógica mediante dos scripts operativos enfocados en la prevención de fugas térmicas y el balance de carga autónomo de la planta. Finalmente, el estudio concluye vinculando las teorías gramaticales analizadas con las observaciones prácticas de rendimiento compilado e interpretado.

Palabras clave: Paradigmas Computacionales, Benchmarking de Rendimiento, Análisis Sintáctico, Lenguaje de Dominio Específico, ECO-GRID

Índice

ÍNDICE	3
INTRODUCCIÓN	4
DESARROLLO	5
Actividad I. Matriz de Paradigmas.	5
Actividad II. Análisis de Lenguajes y Benchmarking.	8
Actividad III. Diseño del Lenguaje L.	27
Conclusiones	33
Referencias Bibliográficas.	34

Introducción

En el desarrollo de la ciencia de la computación, el estudio de los lenguajes de programación suele abordarse inicialmente desde una perspectiva puramente aplicativa, donde la sintaxis se concibe como un mero canal de traducción de la lógica algorítmica. En la actualidad, las demandas de la industria tecnológica han impulsado un fenómeno de convergencia multiparadigma, forzando a lenguajes tradicionales de propósito general a integrar abstracciones funcionales, lógicas o concurrentes para mitigar problemáticas complejas como las condiciones de carrera y los efectos colaterales. Este panorama exige que el ingeniero en informática posea la capacidad de evaluar analíticamente cómo los diferentes mecanismos de ejecución desde la compilación nativa bajo optimizaciones de bajo nivel hasta los motores dinámicos con traducción en tiempo de ejecución (JIT) responden ante algoritmos de procesamiento intensivo. La medición empírica del rendimiento o benchmarking, deja de ser una métrica aislada y se convierte en la validación directa de los fundamentos teóricos y estructurales que sustentan cada paradigma informático. Bajo estas premisas, el presente informe de investigación desglosa un marco comparativo interconectado. Inicialmente, se establece la fundamentación teórica de los paradigmas clásicos y emergentes junto a su impacto conceptual en el sistema. Posteriormente, se contrasta dicha teoría con un análisis empírico de rendimiento sobre los entornos diferenciados de Zig, Rust, Python y JavaScript. Finalmente, se demuestra la viabilidad práctica de la abstracción gramatical mediante la especificación formal del Lenguaje L orientado al control del ecosistema de microrredes eléctricas inteligentes ECO-GRID.

Desarrollo

Actividad I. Matriz de Paradigmas.

A continuación, se presenta la matriz que analiza las partes básicas de los modelos tradicionales y nuevos. Esta descripción detallada explica las diferencias en la velocidad de ejecución y el uso de memoria que se observarán en las pruebas prácticas.

Tabla 1. *Matriz Descriptiva de Paradigmas de Programación.*

Paradigmas Clásicos y Emergentes	Gestión del Estado del Sistema	Mutabilidad y Flujo de Control	Gestión de Memoria e Impacto en Efectos Secundarios
Imperativo / Estructural	Explícita y centralizada. El estado cambia a través de asignaciones sucesivas en variables localizadas.	Secuencial determinada. Controlado por estructuras de salto, bucles e instrucciones condicionales explícitas.	Alta mutabilidad. Modificación directa de celdas de memoria. Propenso a efectos secundarios severos si no se aíslan los ámbitos lógicos.
Orientado a Objetos (POO)	Descentralizada y encapsulada. El estado se distribuye en instancias (objetos) que combinan datos y comportamiento.	Basada en mensajes. Determinado por la invocación de métodos y despacho dinámico (polimorfismo).	Mutabilidad restringida por interfaces. La herencia y composición gestionan las jerarquías de acceso a los recursos.
Funcional	Inexistente de forma global. El estado se deriva conceptualmente a través de la evaluación de expresiones lógicas.	Evaluación perezosa y recursiva. El flujo es dirigido por funciones como ciudadanos de primer orden.	Inmutabilidad absoluta. Transparencia referencial estricta. Eliminación programática de efectos colaterales en la ejecución.
Lógico / Declarativo	Implícita en base a relaciones. El estado se define mediante un	Abstracción total. El motor de inferencia ejecuta unificación y resolución de cláusulas	Gestión abstracta. No existe noción de punteros ni variables físicas para el

	conjunto de hechos y reglas lógicas.	de Horn automáticamente.	desarrollador de la aplicación.
Concurrente / Actores	Aislamiento estricto. Cada actor posee de manera exclusiva su propio estado local incomunicable.	Asíncrona y reactiva. Flujo gobernado exclusivamente por intercambio de paso de mensajes en buzones.	Protección por diseño lingüístico. Mitigación absoluta de condiciones de carrera al prohibir la memoria compartida.

Análisis del Fenómeno de Convergencia Multiparadigma.

En la actualidad, la forma en que se diseñan los programas de software ha cambiado mucho. Ya no se sigue un enfoque teórico puro, porque los entornos de producción modernos son muy complejos. Las herramientas que se utilizan para desarrollar software han adoptado un diseño que combina diferentes enfoques. Esto se llama convergencia multiparadigma.

Un ejemplo importante es el lenguaje de programación JavaScript. Originalmente, JavaScript se diseñó para ser un lenguaje imperativo, pero ahora también incluye características de la programación funcional. Por ejemplo, se pueden utilizar funciones de orden superior, clausuras y métodos declarativos no mutables como `.map()` y `.filter()`. Esto permite a los desarrolladores evitar la manipulación directa de índices en bucles tradicionales.

De manera similar, el lenguaje de programación Python también ha adoptado características de la programación funcional. Permite la comprensión de listas y expresiones lambda, lo que permite a los desarrolladores aislar el flujo de control tradicional. Por otro lado, lenguajes modernos como Rust están diseñados para combinar la optimización y el control estricto de memoria del modelo imperativo con la expresividad y la seguridad del paradigma funcional. Rust incluye conceptos como la inmutabilidad por defecto, tipos algebraicos de datos y un sistema de propiedades altamente abstracto. Esto no se hace por preferencia estética, sino porque es necesario para garantizar la ausencia de condiciones de carrera y corrupciones de

memoria en tiempo de compilación. Además, esto se logra sin la penalización de rendimiento que introduce un recolector de basura en sistemas de procesamiento intensivo.

Este comportamiento práctico explica directamente por qué las arquitecturas con filosofías asíncronas basadas en eventos o compilaciones directas nativas a bajo nivel, como Zig, difieren mucho en su eficiencia real. Al organizar la lógica del software, la convergencia de varios paradigmas permite al ingeniero informático crear capas con una alta abstracción declarativa, mientras que los controladores subyacentes operan con el rendimiento básico de la programación secuencial.

Actividad II. Análisis de Lenguajes y Benchmarking.

El análisis léxico constituye la fase inicial en la que el intérprete o el compilador descompone el flujo continuo de caracteres del código fuente en unidades pequeñas con significado lógico denominados tokens. Este proceso de tokenización permite limpiar el archivo de elementos redundantes para el sistema y clasificar cada componente según su función semántica. Con el propósito de contrastar las diversas filosofías de diseño de software, se presenta a continuación un estudio morfológico de los cuatro entornos tecnológicos:

Análisis de Zig:

- **Catálogo de Palabras Reservadas:** Gobernado por la premisa de evitar cualquier tipo de comportamiento oculto en el compilador, limita sus palabras clave a un mínimo estricto de aproximadamente 40 palabras. Destacan las declaraciones base (const, var, fn, pub, usingnamespace), tipos y estructuras (struct, enum, union, opaque, error), control de flujo (if, else, switch, while, for, break, continue, return, unreachable), manejo de errores y punteros (try, catch, allowzero, noalias, inline) y sus cláusulas insignias de control de recursos y metaprogramación: comptime, defer, errdefer y test.
- **Semántica de Literales:** Excluye cualquier tipo de conversión implícita u oculta de datos. Los enteros y flotantes admiten guiones bajos y formatos de base decimal, hexadecimal, octal y binario. Utiliza un tipo de cadena abstracta y maneja las comillas dobles como punteros constantes a arreglos de bytes con terminación en cero para ser compatible directamente con C. Para bloques multilínea descarta los caracteres de escape y propone el uso de dos barras

invertidas al inicio de cada línea. Los booleanos se limitan a true o false.

- **Elementos Irrelevantes:** Los espacios en blanco entre operadores y los saltos de línea internos son descartados en la tokenización. Zig no posee o soporta comentarios multilínea tradicionales (`/* */`); toda anotación debe ser de línea mediante doble barra (`//`). Emplea comentarios de documentación de tres barras (`///`) para funciones y de doble barra con exclamación (`///!`) para la descripción estructural del archivo completo.
- **Delimitación de Bloques y Sentencias:** Sigue un diseño de sintaxis libre orientado al uso obligatorio de llaves para agrupar funciones y bucles, exigiendo la conclusión formal de cada instrucción independiente con un punto y coma (;). La indentación no altera el flujo lógico y sirve únicamente para la legibilidad humana, estilizada por defecto a 4 espacios. Su compilador aplica un parsing lineal de alta velocidad debido a la ausencia de reglas de inferencia complejas en la separación de sentencias.

Análisis de Python:

- **Catálogo de palabras reservadas:** Cuenta con un catálogo restringido de aproximadamente 35 palabras clave orientadas a la claridad del flujo del programa. Estas se organizan según su función lógica, en control de flujo (if, elif, else, for, while, break, continue, pass), gestión de excepciones (try, except, finally, raise, assert), lógica booleana (and, or, not, in, is), funciones o clases (def, class, return, lambda, yield), manejo de variables (del, global, nonlocal), módulos (import, from, as) y gestión de asincronismo y contexto (with, async, await).
- **Reglas de Identificadores:** Permite caracteres alfanuméricos (letras mayúsculas o minúsculas de la a-z, A-Z y números del 0-9) y guiones bajos. El inicio debe ser estrictamente una letra o un guión bajo, nunca un dígito numérico. Posee sensibilidad estricta a mayúsculas y minúsculas, prohibiendo el uso de espacios en blanco intermedios y caracteres especiales (como !, @, \$). No soporta mecanismos para usar palabras reservadas como identificadores del usuario.
- **Semántica de Literales:** Soporta enteros (admitiendo guiones bajos como separadores visuales, ej: 1_000_000), flotantes y complejos mediante el sufijo j (ej: 5 + 2j). Las cadenas de texto se delimitan por comillas simples, dobles o triples para bloques multilínea. Los literales booleanos se escriben con mayúscula inicial (True, False), emplea None para denotar la ausencia de valor, y posee literales de colección como listas ([]), tuplas (()), diccionarios ({clave: valor}) y conjuntos ({}).
- **Elementos Irrelevantes:** Los espacios en blanco horizontales intermedios entre operadores y variables (como `x = 5 + 3` frente a `x=5+3`) son completamente

ignorados por el intérprete. Los comentarios omiten cualquier cadena de caracteres que se ubique posterior al símbolo numeral (#). Asimismo, permite la división e hilado implícito de líneas extensas cuando los elementos se encuentran contenidos dentro de paréntesis (), corchetes [] o llaves {}.

- **Delimitación de Bloques y Sentencias:** Python destaca por implementar una sintaxis espacial estricta, en la cual los bloques de código se definen exclusivamente por su indentación y no por elementos tipográficos como las llaves. El fin de una instrucción se marca mediante el token de salto de línea, mientras que las tabulaciones o espacios al inicio de cada línea se procesan como tokens de apertura (INDENT) o cierre (DEDENT). El estándar de la comunidad (PEP 8) prescribe el uso de 4 espacios por nivel. Los bloques terminan de forma implícita cuando la indentación regresa a su nivel anterior (desindentación).

Análisis de Rust:

- **Catálogo de Palabras Reservadas:** Altamente restrictivo y enfocado en la seguridad y gestión estática de recursos. Sus palabras clave estrictas regulan declaraciones, mutabilidad y ámbitos inseguros (`let`, `fn`, `struct`, `enum`, `union`, `trait`, `impl`, `const`, `static`, `mut`, `pub`, `mod`, `use`, `as`, `type`, `where`), control de flujo (`if`, `else`, `match`, `loop`, `while`, `for`, `in`, `break`, `continue`, `return`) y asincronismo (`unsafe`, `async`, `await`). Reserva vocablos para adiciones futuras (`box`, `macro`, `do`, `try`, `typeof`, `unsized`, `virtual`). Permite el uso de palabras reservadas como nombres de variables mediante identificadores crudos anteponiendo el prefijo `r#` (ej: `r#match`)
- **Reglas de Identificadores:** Compuestos por caracteres alfanuméricos y guiones bajos, debiendo comenzar obligatoriamente por una letra o un guión bajo. No se permiten espacios ni caracteres especiales, y aplica una rigurosa diferenciación de mayúsculas y minúsculas.
- **Semántica de Literales:** Refleja un sistema de tipos estático y estricto. Admite enteros y flotantes con guiones bajos intermedios, soportando sufijos explícitos de tamaño de bits y signo (ej: `42u32` para enteros de 32 bits sin signo, `10i8` para enteros de 8 bits con signo, o `3.14f64`). Permite bases hexadecimal, octal y binaria. Diferencia un carácter Unicode de 4 bytes en comillas simples (`'ñ'`) de una cadena de texto en comillas dobles, soportando además cadenas crudas (`r#"texto"#`) y literales de bytes anteponiendo la letra `b` (ej: `b'A'` o `b"abc"`). Los booleanos son estrictamente `true` o `false`.
- **Elementos Irrelevantes:** Los espacios de separación intermedios entre variables y operadores se descartan por completo durante la etapa de tokenización. Rust

elimina los comentarios ordinarios de una línea (//) y de bloque (/* */), pero añade comentarios de documentación especial mediante triple barra (///) o barra doble y exclamación (//!), los cuales el compilador procesa para generar manuales técnicos en HTML

- **Delimitación de Bloques y Sentencias:** Se rige bajo una sintaxis libre donde las llaves delimitan de forma obligatoria los bloques de código y los ámbitos de validez de las variables. Cada expresión de asignación o ejecución debe finalizar con un punto y coma (;). La omisión del punto y coma al final de un bloque altera radicalmente la semántica, transformando la expresión en un valor de retorno implícito para la función o ámbito contenedor. La indentación es puramente estética y es estandarizada a 4 espacios por la herramienta rustfmt.

Análisis de JavaScript:

- **Catálogo de Palabras Reservadas:** Su catálogo de palabras clave es extenso. Incluye términos para la gestión y declaración de variables (var, let, const), control de flujo y manejo de errores (if, else, switch, case, default, for, while, do, break, continue, return, try, catch, finally, throw), programación orientada a objetos (class, constructor, extends, super, this, new), asincronismo y funciones (function, function*, yield, async, await), módulos (import, export, from, as), operadores (typeof, instanceof, in, delete, void) y palabras reservadas para el futuro o modos estrictos (package, private, protected, public, interface, implements, static).
- **Reglas de Identificadores:** Permite caracteres alfanuméricos, el guión bajo y, a diferencia de los otros lenguajes, el signo de dólar (\$). Un identificador válido puede comenzar legalmente con una letra, un guión bajo o el signo de dólar, pero nunca con un número. Aplica de forma estricta la sensibilidad a mayúsculas y minúsculas, y prohíbe caracteres especiales como guiones medios o puntos.
- **Semántica de Literales:** Unifica los números enteros y flotantes bajo un mismo tipo primitivo (Number), admitiendo el uso de guiones bajos intermedios. Para enteros de precisión arbitraria (BigInt), exige el sufijo n al final del número (ej: 9007199254740991n). Las cadenas aceptan comillas simples, dobles y saltos de línea. Los booleanos se escriben en minúsculas (true, false), maneja los especiales null (ausencia intencional) y undefined (valor no inicializado), y soporta estructuras literales como objetos ({}), arreglos ([]) y expresiones regulares entre barras (/.../).
- **Elementos Irrelevantes:** Los espacios en blanco al inicio de la línea y los

espacios adicionales intermedios entre operadores son completamente descartados por el motor de ejecución. Soporta comentarios de una sola línea mediante la doble barra (`// comentario`) y comentarios multilínea o de bloque delimitados por los operadores tradicionales `/* comentario */`.

- **Delimitación de Bloques y Sentencias:** JavaScript adopta un enfoque de sintaxis libre, delegando la estructuración de bloques lógicos al uso explícito de llaves y volviendo la indentación un factor puramente estético. Las instrucciones independientes se separan formalmente con puntos y comas (;). No obstante, implementa el mecanismo de Inserción Automática de Puntos y Comas (ASI). Esta característica permite que el motor del lenguaje deduzca e inserte los signos de puntuación omitidos al cambiar de línea. Sin embargo, si el programador se confía y los olvida, el sistema puede malinterpretar el código y generar fallos inesperados en ciertos escenarios de ejecución.

Benchmarking de Conjetura de Collatz

Para llevar a cabo el benchmarking de procesamiento intensivo, se ha seleccionado la demostración de la Conjetura de Collatz. La cual establece que, para cualquier número entero positivo N , la secuencia eventualmente siempre alcanzará el valor de 1 si se aplican repetidamente las siguientes operaciones:

- Si el número es par, se divide entre 2.
- Si el número es impar, se multiplica por 3 y se le suma 1.

Para garantizar una carga de estrés computacional válida, el programa no evalúa un único valor aislado. Por el contrario, la lógica implementada calcula de forma iterativa y continua el total de pasos requeridos para todos los números estrictamente menores al límite N ingresado por el usuario, cumpliendo con la restricción base de $N > 50$.

Habiendo establecido el comportamiento del algoritmo escogido, se presenta seguidamente el desglose del código fuente implementado en cada uno de los lenguajes evaluados, acompañado de su respectivo análisis morfosintáctico.

CÓDIGO FUENTE DE PYTHON

```
import time

def collatz(num):
    pasos = 0
    while num > 1:
        if num % 2 == 0:
            num //= 2
        else:
            num = 3 * num + 1
        pasos += 1
    return pasos

def pedir_numero():
    while True:
        respuesta = input("Ingresa un numero entero positivo mayor a 50: ")
        try:
            n = int(respuesta)
            if n <= 50:
                print("Error, el numero debe ser mayor a 50.")
            else:
                return n
        except ValueError:
            print("Error, ingresa un numero entero valido mayor a 50")

def ejecutar():
    n = pedir_numero()

    tiempo_inicio = time.perf_counter()
    pasos_totales = 0

    for i in range(1, n):
        pasos_totales += collatz(i)

    tiempo_fin = time.perf_counter()

    print(f"Pasos totales calculados: {pasos_totales}")
    print(f"Tiempo: {(tiempo_fin - tiempo_inicio) * 1000:.4f} ms")

if __name__ == "__main__":
    ejecutar()
```

Análisis Morfológico del código en Python

- **Ausencia de Tokens de Tipo:** En la firma **def collatz(num):**, el token **num** asume su tipo de forma dinámica en tiempo de ejecución.
- **Operador de Coerción Evitada:** En la línea **num //= 2**, el uso del token **//=** es crítico. Si se usara el token estándar **/=**, el intérprete mutaría dinámicamente la variable a un tipo de coma flotante (float), corrompiendo el cálculo matemático.

Análisis Sintáctico código en Python

- **Jerarquía por Indentación:** Carece por completo de llaves delimitadoras.

CÓDIGO FUENTE DE JAVASCRIPT

```
const readline = require('readline');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function collatz(num) {
  let pasos = 0;

  while (num > 1) {
    if (num % 2 === 0) {
      num /= 2;
    } else {
      num = 3 * num + 1;
    }
    pasos++;
  }

  return pasos;
}

function pedirNumero() {
  return new Promise((resolve) => {

    rl.question('Ingresa un número entero positivo: ', (respuesta) => {

      if (isNaN(respuesta) || parseInt(respuesta) <= 50) {
        console.log(isNaN(respuesta) ? 'Error: ingresar un número válido.' : 'Error: El numero debe ser mayor a 50.');
```

valido.' : 'Error: El numero debe ser mayor a 50.');

```
        resolve(pedirNumero());

      } else {
        resolve(respuesta);
      }
    });
  });
}

async function ejecutar() {
```

```

const intN = parseInt(await pedirNumero());

const tiempo_inicio = performance.now();

let pasos_totales = 0;

for (let i = 1; i < intN; i++) {
    pasos_totales += collatz(i);
}

const tiempo_fin = performance.now();

console.log(`Pasos totales calculados: ${pasos_totales}`);
console.log(`Tiempo: ${(tiempo_fin - tiempo_inicio).toFixed(4)} ms`);

rl.close();
}

ejecutar();

```

Análisis Morfológico del Código en JavaScript

- **Ámbito Léxico Estricto:** Al inicializar la variable contadora se usa el token **let pasos = 0**. Esto restringe el alcance de la variable exclusivamente al bloque en el que fue declarada, a diferencia de la antigua flexibilidad de **var**.
- **Identidad vs. Igualdad:** En la condicional **if (num % 2 === 0)**, el token **===** exige una evaluación de identidad estricta. El motor V8 no solo verifica el valor matemático, sino que asegura en la fase léxica que ambos operandos sean numéricos antes de proceder.

Análisis Sintáctico del Código en JavaScript

- **Delimitación Clásica:** Al estructurar la lógica algorítmica, el manejo de bloques encapsulados por llaves **{}** y el uso de paréntesis obligatorios en **while (num > 1)** preserva una herencia sintáctica muy similar a la que se utiliza en entornos estructurados como Java o C++, lo que evita ambigüedades en la lectura del motor JIT.

CÓDIGO FUENTE DE RUST

```
// Grupo Nabla
// Conjetura de Collatz en Rust
// para todos los números menores a N, con N ingresado por el usuario

use std::io::{self, Write};
use std::time::Instant;
use std::hint::black_box;

fn collatz(mut num: u64) -> u64 {
    let mut pasos = 0;
    while num > 1 {
        if num % 2 == 0 {
            num /= 2;
        } else {
            num = 3 * num + 1;
        }
        pasos += 1;
    }
    pasos
}

fn pedir_numero() -> u64 {
    loop {
        print!("Ingresa un numero entero positivo mayor a 50: ");
        io::stdout().flush().unwrap();

        let mut input = String::new();
        io::stdin().read_line(&mut input).unwrap();

        match input.trim().parse::<u64>() {
            Ok(n) if n > 50 => return n,
            Ok(_) => println!("Error, el numero debe ser mayor a 50."),
            Err(_) => println!("Error, ingresa un numero valido."),
        }
    }
}

fn main() {
    let n = pedir_numero();

    let tiempo_inicio = Instant::now();
    let mut pasos_totales = 0;
```

```

for i in 1..n {
    pasos_totales += collatz(i);
}

// Evita que el compilador LLVM optimice la variable
black_box(pasos_totales);

let tiempo_fin = tiempo_inicio.elapsed();

println!("Pasos totales calculados: {}", pasos_totales);
println!("Tiempo: {:.4} ms", tiempo_fin.as_secs_f64() * 1000.0);
}

```

Análisis Morfológico del Código en Rust

- **Inmutabilidad por Defecto:** En la firma `fn collatz(mut num: u64) -> u64`, el token **mut** es obligatorio. Rust asume inmutabilidad absoluta; sin este token, el compilador arrojaría un error léxico al intentar reasignar el valor de **num** dentro del bucle.
- **Token de Retorno:** El token `->` antes de **u64** indica explícitamente al compilador el tamaño en memoria exacto del tipo de dato de salida.

Análisis Sintáctico del Código en Rust

- **Orientación a Expresiones (Retorno Implícito):** La última línea de la función es simplemente la palabra `pasos` sin punto y coma (`;`). En la gramática de Rust, la ausencia del delimitador “`;`” al final de un bloque transforma la línea en un retorno de valor directo, eliminando la necesidad de la palabra reservada **return**.
- **Jerarquía del Rango:** En el programa principal, el iterador `for i in 1..n { ... }` abstrae la complejidad. No hay declaración de variables de control ni de incrementos manuales; la estructura sintáctica **1..n** genera automáticamente la secuencia límite.

CÓDIGO FUENTE DE ZIG

```
const std = @import("std");

fn collatz(start_num: u64) u64 {
    var num = start_num;
    var pasos: u64 = 0;

    while (num > 1) {
        if (num % 2 == 0) {
            num /= 2;
        } else {
            num = 3 * num + 1;
        }
        pasos += 1;
    }
    return pasos;
}

fn pedirNumero(stdout: anytype) !u64 {
    const stdin = std.io.getStdIn().reader();
    var buf: [100]u8 = undefined;

    while (true) {
        try stdout.print("Ingresa un numero entero positivo mayor a 50: ", .{});

        if (try stdin.readUntilDelimiterOrEof(buf[0..], '\n')) |user_input| {
            const line = std.mem.trimRight(u8, user_input, "\r");

            const n = std.fmt.parseInt(u64, line, 10) catch {
                try stdout.print("Error: ingresar un numero valido.\n", .{});
                continue;
            };

            if (n <= 50) {
                try stdout.print("Error: El numero debe ser mayor a 50.\n", .{});
            } else {
                return n;
            }
        }
    }
}

pub fn main() !void {
    const stdout = std.io.getStdOut().writer();
```

```

const n = try pedirNumero(stdout);

var timer = try std.time.Timer.start();

var pasos_totales: u64 = 0;

for (1..n) |i| {
    pasos_totales += collatz(i);
}

const tiempo_ns = timer.read();

std.mem.doNotOptimizeAway(pasos_totales);

// convertir tiempo en Ns a Ms
const tiempo_ms = @as(f64, @floatFromInt(tiempo_ns)) / 1_000_000.0;

try stdout.print("Pasos totales calculados: {d}\n", .{pasos_totales});

try stdout.print("Tiempo preciso (decimales): {d:.4} ms\n", .{tiempo_ms});
}

```

Análisis Morfológico (Léxico)

- **Protección de Parámetros:** En la firma `fn collatz(start_num: u64) u64`, el token `start_num` es estrictamente inmutable por diseño del lenguaje. Esto obliga a crear una copia explícita en memoria usando el token de mutabilidad: `var num = start_num;`
- **Manejo de Errores como Expresiones:** En la entrada de datos, el token `catch`, en la línea `catch { continue; }`, atrapa los fallos de conversión de texto a número directamente en la asignación, en lugar de envolver todo el bloque en complejas estructuras `try-except` como en Python.

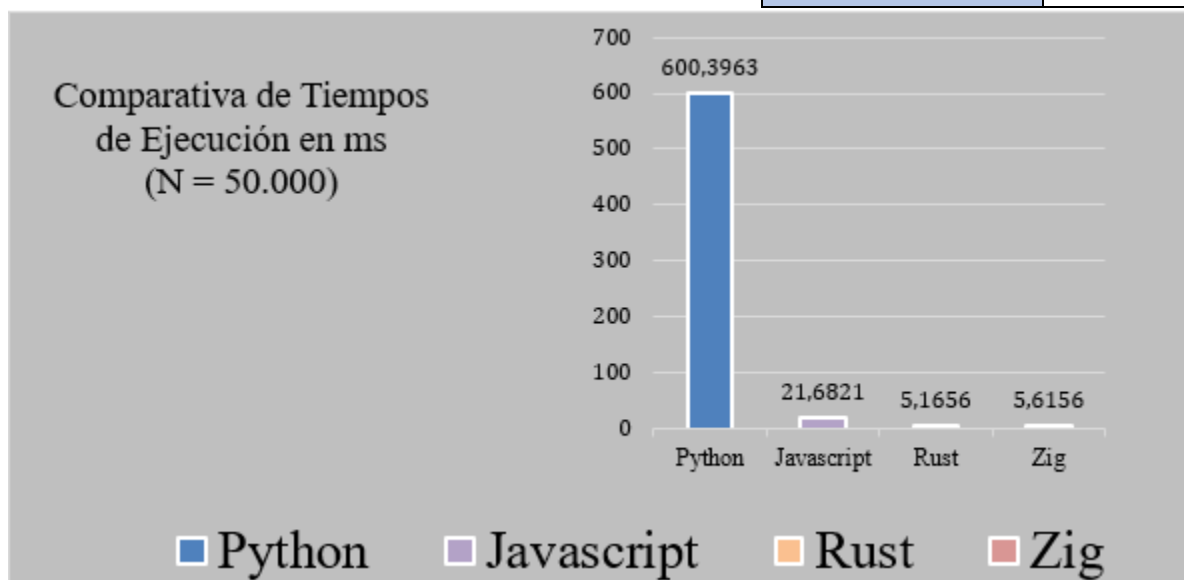
Análisis Sintáctico

- **Captura de Carga Útil** : Al ejecutar el bucle que llama a la función, la sintaxis **for (1..n)** |i| utiliza tuberías | | . Esta estructura gramatical única extrae el valor actual de la iteración y lo ata directamente al identificador local **i** de forma segura, evitando desbordamientos durante el procesamiento continuo que demanda la CPU al ejecutar el algoritmo.

Resultados del Benchmarking o Prueba de Rendimiento

Datos del Entorno de Pruebas	
Sistema Operativo	Windows 11 Pro
CPU	Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz (1.80 GHz)
Memoria RAM	16 GB

Lenguaje	Tiempo en ms
Rust	5,1656 ms
Zig	5,6156 ms
Javascript	21,6821 ms
Python	600,3963 ms



Tras analizar estos resultados, ordenados de manera ascendente, podemos notar que indiscutiblemente Rust se posiciona como el ganador en esta ocasión, lo que evidencia y valida

cómo las decisiones del diseño léxico y sintáctico en cada lenguaje impactan directamente en el rendimiento del hardware. Asimismo, es posible observar cómo la flexibilidad y velocidad de escritura que Python ofrece al desarrollador tiene un costo computacional severo: al carecer de tokens de tipado estricto, obliga a la máquina a inferir y validar dinámicamente, en cada iteración, si una variable es un entero antes de aplicar una operación matemática, sacrificando así la velocidad de procesamiento.

Es importante destacar que el control riguroso que tanto Rust como Zig nos exigen tener a la hora de programar como, por ejemplo, declarar explícitamente la mutabilidad, definir tipos de datos estrictos y manejar los estados de manera manual, le ahorra todo este trabajo de inferencia al procesador. Esto se traduce en un rendimiento inigualable que ha quedado sólidamente evidenciado en las métricas de esta prueba.

Actividad III. Diseño del Lenguaje L.

1. Especificación del Alfabeto y Reglas Léxicas

Para garantizar un análisis léxico libre de ambigüedades en el compilador conceptual, se definen los siguientes componentes fundamentales:

- **Alfabeto:** El conjunto de caracteres válidos está restringido a letras del alfabeto latino (**a-z, A-Z**), dígitos numéricos (**0-9**), el carácter de subrayado (**_**) y los siguientes símbolos delimitadores y operadores: **(,), ,, =, >, <**, y la palabra reservada clave **Y** como operador lógico.
- **Reglas de Identificadores:** Los nombres de variables o identificadores de dispositivos deben comenzar obligatoriamente con una letra minúscula, seguida de letras, números o guiones bajos (Ej: temp_bateria1, estado_reles).
- **Literales Soportados:**
 - **Numéricos:** Enteros y de punto flotante para lecturas analógicas (Ej: 10 , 10.5).
 - **Booleanos:** Representados estrictamente por las palabras reservadas VERDADERO y FALSO.
- **Tratamiento de Elementos Irrelevantes:** Los espacios en blanco, tabulaciones y saltos de línea actúan únicamente como separadores de tokens y son ignorados por el analizador léxico. Los bloques de código no dependen de la indentación, en su lugar, utilizan tokens unificados con guión bajo (**FIN_SI** y **FIN_MIENTRAS**), para delimitar explícitamente el alcance de las estructuras.

2. Palabras Clave y Primitivas Obligatorias

El lenguaje cuenta con un conjunto de palabras reservadas e instrucciones nativas diseñadas específicamente para la abstracción del hardware de la microred ECO-GRID:

- **Palabras de Control de Flujo y Lógica:** SI, ENTONCES, SINO, FIN_SI, MIENTRAS, EJECUTAR, FIN_MIENTRAS, Y.

- **Primitivas del Sistema:**
 - **init_grid:** Inicializa los controladores de bajo nivel y los canales de comunicación de la microred.
 - **leer_temperatura(id):** Devuelve un valor numérico flotante en grados Celsius proveniente del sensor térmico de la celda especificada.
 - **estado_carga(id):** Devuelve el porcentaje actual de almacenamiento de un banco de baterías específico (0 a 100).
 - **leer_flujo(id):** Devuelve un valor numérico que representa la medición analógica de entrada o salida en kilovatios (kW) desde los sensores de los inversores.
 - **conmutar_linea(id, estado):** Envía una señal al relé de alta potencia para conectar (VERDADERO) o aislar (FALSO) el sector de consumo indicado.
 - **alerta_emergencia(mensaje):** Emite una señal síncrona de interrupción en la interfaz HMI para advertir al operador.

3. Gramática Sintáctica Abstracta

Para que el analizador sintáctico valide las sentencias y construya el árbol de análisis sintáctico (AST) de forma determinista y sin ambigüedades, se define la siguiente gramática en notación BNF formal expandida:

<Sentencia_Condicional>	"SI" <Condicion> "ENTONCES" <Sentencias> "FIN_SI" "SI" <Condicion> "ENTONCES" <Sentencias> "SINO" <Sentencias> "FIN_SI"
<Sentencia_Iterativa>	"MIENTRAS" <Condicion> "EJECUTAR" <Sentencias> "FIN_MIENTRAS"
<Asignación>	<Identificador> "=" <Expresión>
<Llamada_Primitiva>	<Nombre_Funcion> "(" <Argumentos> ")"
<Condicion>	<Expresión> <Comparador> <Expresión> <Condicion> "Y" <Condicion>
<Expresión>	<Identificador> <Literal_Numerico> <Llamada_Primitiva>
<Comparador>	">" "<" "="
<Argumentos>	<Identificador> <Literal_Numerico> <Identificador> "," <Argumentos>

Scripts de Ejemplo para los Escenarios Operativos en Lenguaje L

- **Escenario Operativo A:** Prevención de Fuga Térmica y Gestión de Alivio de Carga

Propósito: Monitorear iterativamente la temperatura del banco de baterías de litio BAT_01. Si supera los 55°C, activa la refrigeración, desconecta los paneles solares de carga y desvía la fábrica a la red comercial de respaldo.

```
init_grid

// Se asume un ciclo de monitoreo continuo interactuando con el
hardware

MIENTRAS VERDADERO EJECUTAR

    t_actual = leer_temperatura(BAT_01)

    SI t_actual > 55.0 ENTONCES

        // Activar sistemas de refrigeración auxiliar
        conmutar_linea(REFRI_AUX, VERDADERO)

        // Desconectar líneas de carga solar para detener el ingreso
término
        conmutar_linea(PANEL_SOLAR_IN, FALSO)

        // Desviar el consumo del sector industrial a la red comercial
de respaldo
        conmutar_linea(SECTOR_INDUSTRIAL_LOCAL, FALSO)
        conmutar_linea(RED_COMERCIAL_BACKUP, VERDADERO)

        // Emitir alerta recurrente mientras persista la condición
crítica
        alerta_emergencia("CRÍTICO: Fuga térmica detectada en BAT_01.
Protocolo de alivio activado.")

    SINO
```

```

        // Si la temperatura se normaliza por debajo del umbral de
seguridad

        SI t_actual < 45.0 ENTONCES

            conmutar_linea(REFRI_AUX, FALSO)

            conmutar_linea(PANEL_SOLAR_IN, VERDADERO)

            conmutar_linea(SECTOR_INDUSTRIAL_LOCAL, VERDADERO)

            conmutar_linea(RED_COMERCIAL_BACKUP, FALSO)

        FIN_SI

    FIN_SI

FIN_MIENTRAS

```

- **Escenario Operativo B:** Balance de Carga y Optimización Energética Autónoma

Propósito: Evaluar el estado de carga. Si es óptimo ($>90\%$) y hay excedente solar detectado por los inversores, vende energía a la red pública. Si cae críticamente ($<20\%$) durante horas de alta demanda nocturna, aísla sectores no esenciales para proteger las áreas médicas y de servidores críticos.

```

init_grid

carga_bat = estado_carga(BAT_01)

produccion_solar = leer_flujo(INVERSOR_SOLAR)

demanda_interna = leer_flujo(INVERSOR_INTERNO)

// CONDICIÓN 1: Excedente óptimo - Inyección a la red pública

```

```
SI carga_bat > 90 Y produccion_solar > demanda_interna ENTONCES

    conmutar_linea(RELE_INYECCION_PUBLICA, VERDADERO)

    alerta_emergencia("OPTIMIZACIÓN: Inyectando excedente
energético a la red general.")

SINO

    conmutar_linea(RELE_INYECCION_PUBLICA, FALSO)

FIN_SI


// CONDICIÓN 2: Estado crítico nocturno - Aislamiento de cargas
no esenciales

SI carga_bat < 20 ENTONCES

    // Aislar sectores no esenciales de la fábrica (ej. oficinas,
almacén)

    conmutar_linea(SECTOR_FABRICA_NO_ESENCIAL, FALSO)

    // Garantizar y preservar el suministro en áreas críticas de
la organización

    conmutar_linea(SECTOR_AREAS_MEDICAS, VERDADERO)

    conmutar_linea(SECTOR_SERVIDORES_CRITICOS, VERDADERO)

    alerta_emergencia("SISTEMA EN RACIONAMIENTO: Sectores no
esenciales aislados. Batería baja.")

FIN_SI
```

Conclusiones

El desarrollo de este trabajo demuestra que el rendimiento de un programa depende directamente de cómo está diseñado su lenguaje de programación. En la Actividad I, se ha podido notar que los lenguajes actuales ya no siguen una sola regla pura, sino que combinan lo mejor de varios estilos para que los códigos sean más fáciles de escribir sin perder el control del sistema.

Esto lo confirmamos con las pruebas de tiempo en la Actividad II. Lenguajes como Rust y Zig son increíblemente rápidos (apenas 5 milisegundos) porque obligan al programador a ser sumamente ordenado con el tipo de datos y el uso de la memoria. Como todo está aclarado desde el principio, la computadora no pierde tiempo adivinando y ejecuta las operaciones de inmediato. En cambio, Python, al ser más flexible y permisivo, hace que la computadora trabaje el triple revisando cada dato en cada iteración del ciclo, lo que frena considerablemente su velocidad con volúmenes grandes de información.

Finalmente, la Actividad III nos permitió aplicar toda esta teoría al crear el Lenguaje L. Al diseñar nuestras propias palabras clave y reglas exactas, demostramos cómo se puede construir un lenguaje especializado que sea seguro, fácil de entender y que responda sin errores ni retrasos a las órdenes de una red eléctrica en tiempo real.

Referencias Bibliográficas.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Ciudad de México, México: Pearson Educación.

ECMA International. (2025). *ECMAScript 2025 language specification* (16th ed.). Recuperado de <https://tc39.es/ecma262/>

Flanagan, D. (2020). *JavaScript: The definitive guide* (7th ed.). Sebastopol, CA: O'Reilly Media.

Geomaths. (2024, 23 Junio). *Collatz conjecture*. Recuperado de

<https://geomaths.co.uk/2024/06/23/collatz-conjecture/>

JavaScript | MDN. (2026, 22 mayo). Recuperado de [https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/JavaScript)

[US/docs/Web/JavaScript](https://developer.mozilla.org/en-US/docs/Web/JavaScript)

Louden, K. C. (2004). *Lenguajes de programación: Principios y práctica* (2da ed.). Ciudad de México, México: Thomson.

Márquez, F. (2026). *Tema 2: Los lenguajes de programación. Asignación II: Informe de investigación y defensa* [Guía de estudio no publicada]. Coordinación de Ingeniería en Informática, Universidad Nacional Experimental de Guayana, Puerto Ordaz, Venezuela.

Matsakis, N. D., & Klock, F. S. (2014). The Rust language. *ACM SIGAda Ada Letters*, 34(2), 103-104. doi:10.1145/2692956.2663188

Python Software Foundation. (2026). *The Python language reference* (v3.12). Recuperado de <https://docs.python.org/3/reference/>

Sebesta, R. W. (2012). *Concepts of programming languages* (10th ed.). Boston, MA: Pearson.

The Rust programming language - The Rust programming language. (s. f.). Recuperado de <https://doc-rust->

[laorg.translate.google/book/? x tr sl=en& x tr tl=es& x tr hl=es& x tr pt=sg](https://doc-rust-)

Van Rossum, G., & Drake, F. L. (2009). *Reglas de estilo para el código Python: PEP 8*.

Recuperado de <https://peps.python.org/pep-0008/>

Watt, D. A. (2004). *Programming language design concepts*. Chichester, Inglaterra: John Wiley & Sons.

Zig Software Foundation. (2026). *Zig language reference documentation*. Recuperado de

<https://ziglang.org/documentation/general/>